1. this Keyword in Java

Definition

`this` is a reference variable that refers to the current object of the class.

It is mainly used to differentiate instance variables from local variables.

Why Do We Need `this`?

When constructor parameters have the same name as instance variables, Java refers to the local variable by default.

Problem Without `this`

```
class Student {
  int id;

  Student(int id) {
     id = id;   // Refers to local variable
  }
}
```

Here, the instance variable is not initialized.

Correct Version Using `this`

```
class Student {
  int id;

  Student(int id) {
     this.id = id;
```

```
    }

    void display() {
        System.out.println("ID: " + id);
    }

    public static void main(String[] args) {
        Student s1 = new Student(101);
        s1.display();
    }
}
```

Output:

ID: 101

`this` always points to the current object in heap memory.

Uses of `this`

Use	Example
Refer instance variable	this.name = name;
Call current class method	this.display();
Call current constructor	this(101);
Pass current object	method(this);

Note: `this()` must be the first statement in a constructor.

---

2. super Keyword in Java

Definition

`super` refers to the immediate parent class object.

Used mainly in inheritance.

---

super Variable Example

```
class Parent {
  int x = 100;
}

class Child extends Parent {
  int x = 200;

  void show() {
     System.out.println("Child x: " + x);
     System.out.println("Parent x: " + super.x);
  }

  public static void main(String[] args) {
     Child obj = new Child();
     obj.show();
  }
}
```

Output:

```
Child x: 200
Parent x: 100
```

---

super Method Example

```
class Parent {
  void show() {
```

```
        System.out.println("Parent method");
    }
}

class Child extends Parent {
    void show() {
        super.show();
        System.out.println("Child method");
    }

    public static void main(String[] args) {
        Child obj = new Child();
        obj.show();
    }
}
```

---

super() Constructor Example

```
class Parent {
    Parent() {
        System.out.println("Parent Constructor");
    }
}

class Child extends Parent {
    Child() {
        super();
        System.out.println("Child Constructor");
    }

    public static void main(String[] args) {
        Child obj = new Child();
    }
}
```

Important Points:

- `super()` is automatically called if not written.
- It must be the first statement in the constructor.

Difference Between this and super

Feature	this	super
Refers to	Current object	Parent object
Used in	Same class	Child class
Constructor call	this()	super()
Used in inheritance	No	Yes

---

3. Exception Handling in Java

What is an Exception?

An Exception is an unwanted or unexpected event that occurs during program execution and disrupts normal flow.

In Java, exceptions are objects.

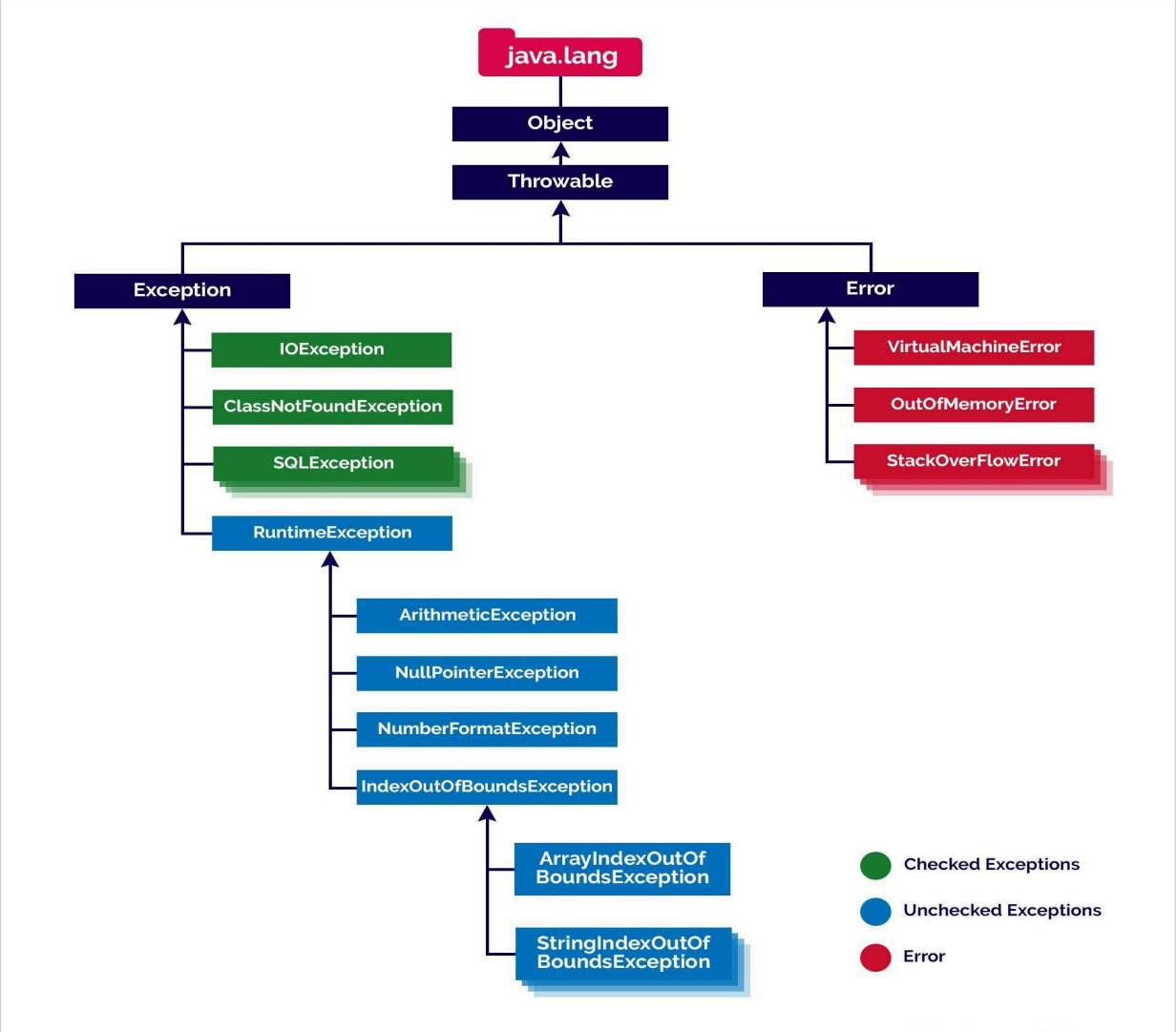
Example Without Exception Handling

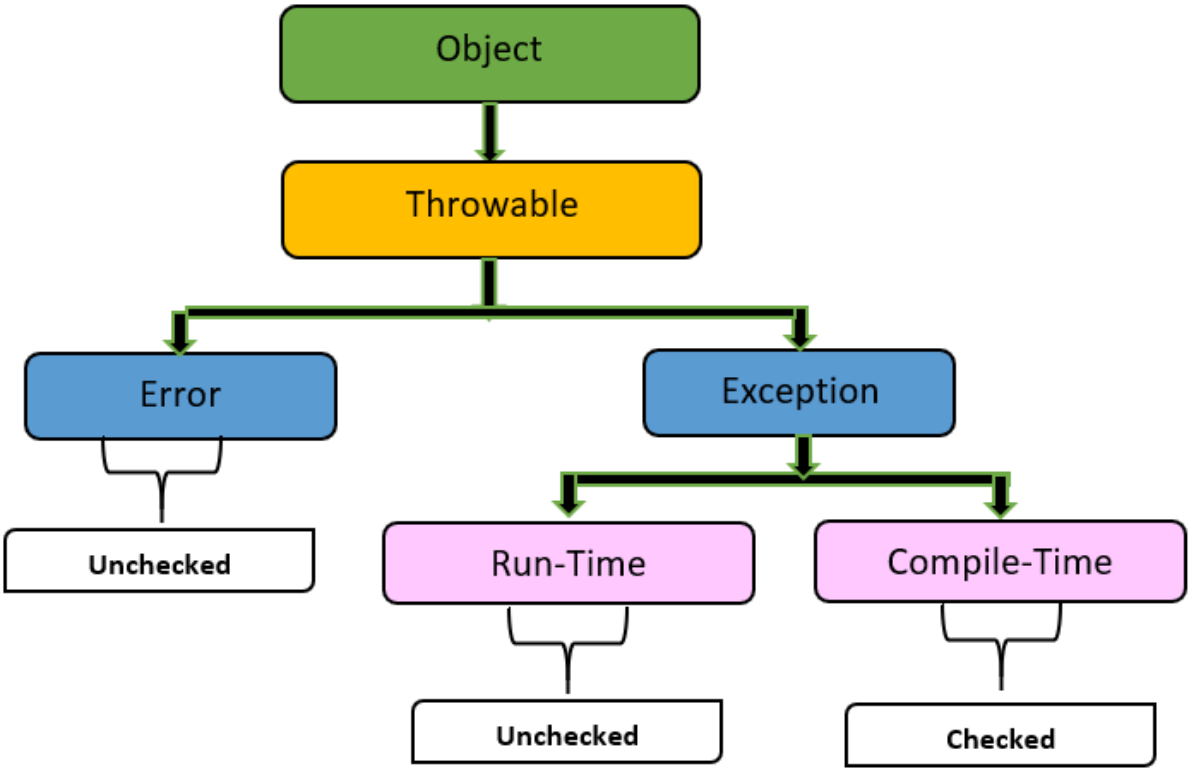
```
class Demo {
  public static void main(String[] args) {
      int a = 10;
      int b = 0;
      int c = a / b;   // ArithmeticException
      System.out.println(c);
```

```
  }
}
```

Program terminates abnormally.

Exception Hierarchy





Main Classes:

- Object
  - Throwable
    - Error
    - Exception

Types:

1. Checked Exception (Compile time)
2. Unchecked Exception (Runtime)

---

try–catch Block

```
class Demo {
  public static void main(String[] args) {
      try {
          int a = 10;
          int b = 0;
          int c = a / b;
      } catch (ArithmeticException e) {
          System.out.println("Cannot divide by zero");
      }

      System.out.println("Program continues...");
  }
}
```

---

finally Block

The finally block always executes whether exception occurs or not.

Used for:

- Closing files
- Closing database connections
- Resource cleanup

---

throw Keyword

Used to manually throw an exception.

```
if(age < 18) {
   throw new ArithmeticException("Not eligible");
}
```

---

throws Keyword

Used in method signature to declare exception.

```
static void readFile() throws IOException {
}
```

Custom Exception

```
class InvalidAgeException extends Exception {
   InvalidAgeException(String msg) {
      super(msg);
   }
}
```

---

4. Garbage Collection in Java

Definition

Garbage Collection is the automatic process of destroying unused objects from heap memory.

Java provides automatic memory management.

When Does Object Become Eligible for GC?

1. Object reference set to null
2. Reassigning reference
3. Object created inside method and method ends

Example

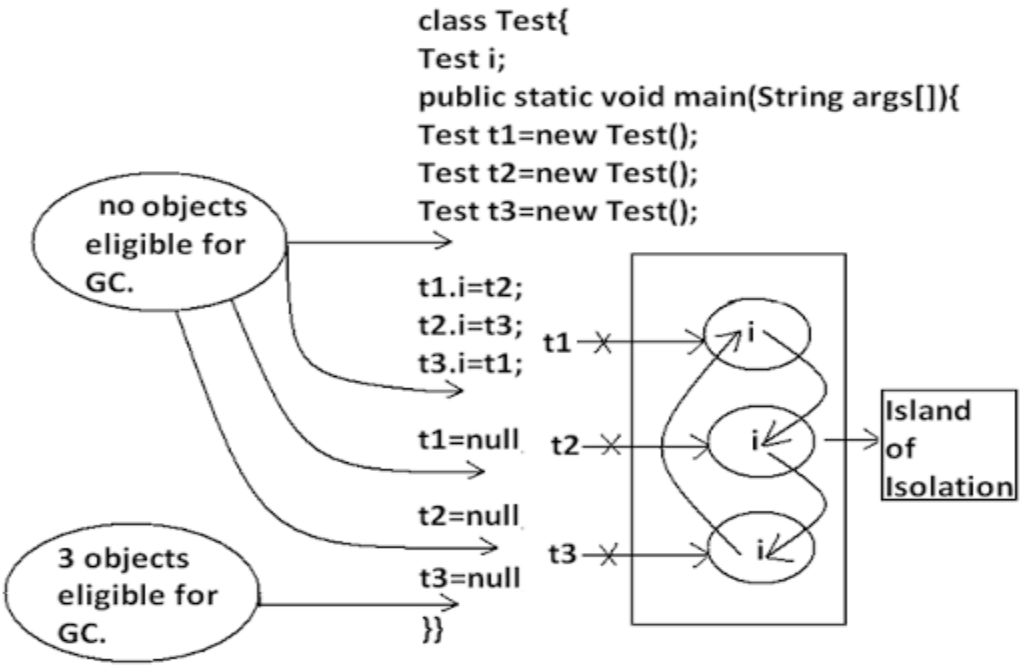
```
class Demo {
  public static void main(String[] args) {
      Demo obj = new Demo();
      obj = null;   // Eligible for GC
  }
}
```

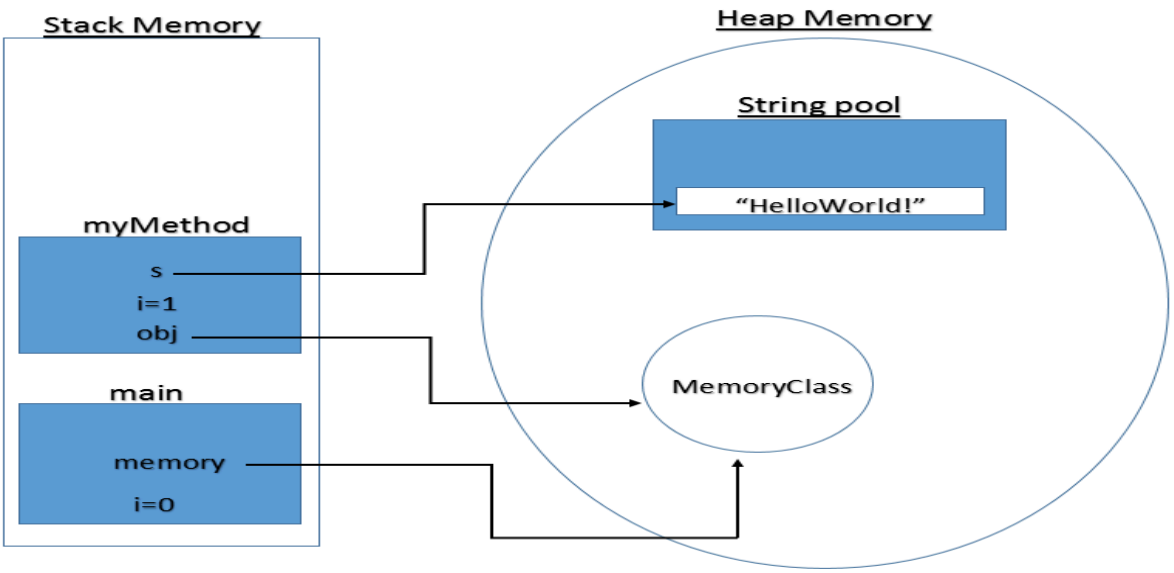
Requesting Garbage Collector

System.gc();

Note: It is only a request, not a guarantee.

Memory Concept (Based on Your Board Diagram)





4

Objects are stored in Heap.
Garbage Collector removes unreferenced objects.

---

5. finalize() Method

Definition

`finalize()` is a method called by Garbage Collector before destroying an object.

It is defined in Object class.

Example

```
class Demo {

  protected void finalize() {
      System.out.println("Finalize method called");
  }

  public static void main(String[] args) {
      Demo obj = new Demo();
      obj = null;

      System.gc();
  }
}
```

Important Points:

- finalize() is not guaranteed to execute.
- It is deprecated in modern Java versions.
- Used earlier for resource cleanup.

Important Interview Points

- `this` cannot be used in static context.
- `super()` must be first statement in constructor.
- Checked exceptions must be handled or declared.
- finally block always executes (except System.exit()).
- Garbage collection is automatic in Java.
- finalize() is deprecated and not reliable.

Constructor Chaining in Java

Definition

Constructor Chaining is the process of calling one constructor from another constructor within the same class or from the parent class, in order to reuse initialization code.

It helps in avoiding code duplication and ensures proper object initialization.

Types of Constructor Chaining

1. Within the Same Class (Using `this()`)

When one constructor calls another constructor of the same class using `this()`.

Example

```
class Student {
  int id;
  String name;

  Student() {
     this(101, "Rahul");   // Calls parameterized constructor
     System.out.println("Default Constructor");
```

```
    }

    Student(int id, String name) {
        this.id = id;
        this.name = name;
        System.out.println("Parameterized Constructor");
    }

    public static void main(String[] args) {
        Student s = new Student();
    }
}
```

Output:

```
Parameterized Constructor
Default Constructor
```

Important Rule:

- `this()` must be the first statement inside the constructor.

---

2. Between Parent and Child Class (Using `super()`)

When a child class constructor calls the parent class constructor using `super()`.

Example

```
class Parent {
    Parent() {
        System.out.println("Parent Constructor");
    }
}

class Child extends Parent {
```

```
    Child() {
        super();   // Calls parent constructor
        System.out.println("Child Constructor");
    }

    public static void main(String[] args) {
        Child obj = new Child();
    }
}
```

Output:

```
Parent Constructor
Child Constructor
```

Important Rules:

- `super()` must be the first statement in the constructor.
- If not written explicitly, Java automatically inserts `super()`.

Why Constructor Chaining is Important

- Ensures proper initialization of objects.
- Improves code reusability.
- Reduces redundancy.
- Maintains constructor execution order (Parent → Child).

2-Mark Exam Definition

Constructor Chaining is the process of calling one constructor from another constructor using `this()` or `super()` to reuse initialization code and ensure proper object creation.